Which RAG Configuration for Which Query?

Mid-submission progress report, Information Retrieval and Extraction, 31 October 2026

Team Agents: Samyak Jain (2024101062), Arnav Agnihotri (2024101103), Sherley Sonali Peethala (2023111022)

Where the project stands

We started out planning to sweep RAG settings and report which ones were worth their cost. The TA pointed out that Wang et al. (2024) and AutoRAG already cover this, so we changed the question. We now run every configuration on every question, which tells us, per question, which configurations worked and what each one cost. We can then ask whether the right configuration can be predicted for a new question, and how much that saves over always using the best fixed configuration. Router A sees only the question. Router B also sees cheap retrieval signals such as the score distribution and the overlap between BM25 and dense results.

After our meeting with the professor we moved to three tasks, each with its own metric: open-domain QA (NQ-open, HotpotQA, TriviaQA; F1), fact verification (FEVER; accuracy) and slot filling (zsRE; exact match). Routers will be trained both globally and per task. The grid has 24 configurations: BM25, dense or hybrid retrieval, top-3 or top-10 passages, reranker on or off, and Qwen2.5 1.5B or 7B. Everything runs on DPR's 100-word Wikipedia passages, so chunking is fixed.

What we have done

Pipeline. The code covers data sampling, the sub-corpus, grouped train/dev/test splits, the grid runner, cost counting, question and retrieval features, the routers and baselines, paired bootstrap, and the tables and figures. AutoRAG normally keeps only the best module at each step, which would leave most of the grid empty, so we call its modules in stages and cache each one.

Pilot on Ada. We ran 300 questions (100 each from the three QA datasets) through all 24 configurations on a 100,000-passage sub-corpus. This took about two hours on one GTX 1080 Ti. That GPU is too old for vLLM, so the pilot used llama.cpp. Two early runs failed (Chroma's batch limits, and AutoRAG crashing on llama.cpp's responses) and both are fixed.

Validity checks. On the two configurations we compared, retrieved passages matched AutoRAG's own end-to-end pipeline exactly, and answers matched 100% and 94%. Our token counts matched the engine's. Re-running one configuration gave 97.3% identical answers, so llama.cpp's parallel batching adds a small amount of noise. No test question shares a gold article with training.

Pilot results. These are from only 60 test questions, so we treat them as proof that the pipeline works and not as findings.

Policy	F1	Cost
Best fixed configuration (hybrid, k=10, no rerank, 7B)	0.591	1,390
Oracle (best configuration per question)	0.787	163
Router A, GBDT	0.584	961
Router B, GBDT	0.575	1,042
Dataset identity only, logistic regression	0.616	832

Costs use placeholder weights. The oracle's gain over the best fixed configuration is +0.196 F1 (95% CI 0.117 to 0.280), but it is uneven: +0.31 on HotpotQA, +0.26 on NQ and only +0.01 on TriviaQA. No router beat the best fixed configuration on quality, and every router's F1 difference has a CI that includes zero. Some match it at lower cost. What worries us is that knowing the dataset does as well as any router. The routers had only 180 training questions, so the full run is the real test. We also dropped a permutation test on the oracle after it gave a null of 0.9999, above the real 0.787. It was destroying question difficulty, so predictability is now judged only by routers on held-out data.

vLLM. A smoke test on an RTX 2080 Ti ran 3.3 to 4.7 times faster than the pilot, and re-runs were 100% identical. The 7B model needs 4-bit weights to fit in 11 GB. There is one open problem: the 1.5B model often answers in full sentences and its F1 drops (0.369 vs 0.393, and 0.427 vs 0.453). We suspect half precision on this GPU but have not confirmed it.

Written but not yet run on the cluster. FEVER and zsRE support, per-task routers and a cost calibration job are all in the repository. Neither new task has been through the grid. The per-task router code has only been run on synthetic data, and the calibration job has not been run at all. All 30 unit tests that run locally pass; two more need Chroma, which is only installed on Ada.

What is left

- **First days of November:** settle the engine by re-running the 1.5B model at full precision and counting garbled 7B answers. Then run the calibration job on that engine and GPU, because the token and call counts are exact but the cost weights are still estimates.
- **By 7 November:** build the 1M-passage sub-corpus and run the full grid on 5 datasets with 1,000 questions each. Extrapolating from pilot timings, this is about 7 GPU-hours on vLLM or 26 on llama.cpp, in 6-hour jobs on one GPU.
- **8 to 14 November:** hand-label 150 answers to validate the LLM judge, choose a judge from outside the Qwen family, and use RAGAS as a secondary metric only if it passes. Train global and per-task routers with 5 seeds, choose the cost-quality trade-off on dev, and run the feature ablations.
- **15 to 19 November:** break results down by task, dataset and question type, do the error analysis, and test sensitivity to cost weights and amortisation. A few tables and figures for this still need code, including the chance-corrected FEVER score.
- **20 and 21 November:** final review and submission.

We also still have to go through the reference lists and citing papers of the four papers the TA flagged. Two points need the professor's confirmation: whether three tasks is what was meant, and whether reporting each metric on its own scale, with averages that weight each task equally, is acceptable as our normalisation. We chose this over z-scores or min-max scaling because those change whenever the set of configurations changes.

The main risk is that routers still cannot beat the fixed configuration or dataset identity on the full data. That would still be a result, since it would help explain why fixed recipes work as well as they do. If time runs short we will cut the latency study first, then the MLP router, RAGAS at scale, TriviaQA, and finally the number of questions per dataset. The three tasks stay.